Computer VisionAI-4002	Lab 08 Object Detection: R-CNN, Fast R-CNN & Faster R-CNN

Course Code: AI-4002		

Contents

1. Objectives
2. Region-Based Convolutional Neural Networks (R-CNN Family)
    - 2.1 R-CNN (Region-Based CNN)
    - 2.2 Fast R-CNN
    - 2.3 Faster R-CNN
        - 2.3.1 Faster R-CNN Algorithm Workflow
    - 2.4 Architectural Comparison Summary
3. Understanding the Problem Statement
4. Data Exploration
    - 4.1 Dataset Structure and Files
    - 4.2 Loading and Inspecting the Annotation CSV
    - 4.3 Inspecting Training Images and Class Distribution
    - 4.4 Visualizing Ground-Truth Bounding Boxes
5. Implementing Faster R-CNN
    - 5.1 Repository Setup and Input Annotation Format
    - 5.2 Converting CSV Annotations to Text Format
    - 5.3 Training the Faster R-CNN Model
    - 5.4 Running Inference and Saving Predictions
6. Lab Task
    - 6.1 Object Detection Using R-CNN

Talha Shahid

1. Objectives

- Understand the architecture and workflow of Regional Convolutional Neural Networks (R-CNN).

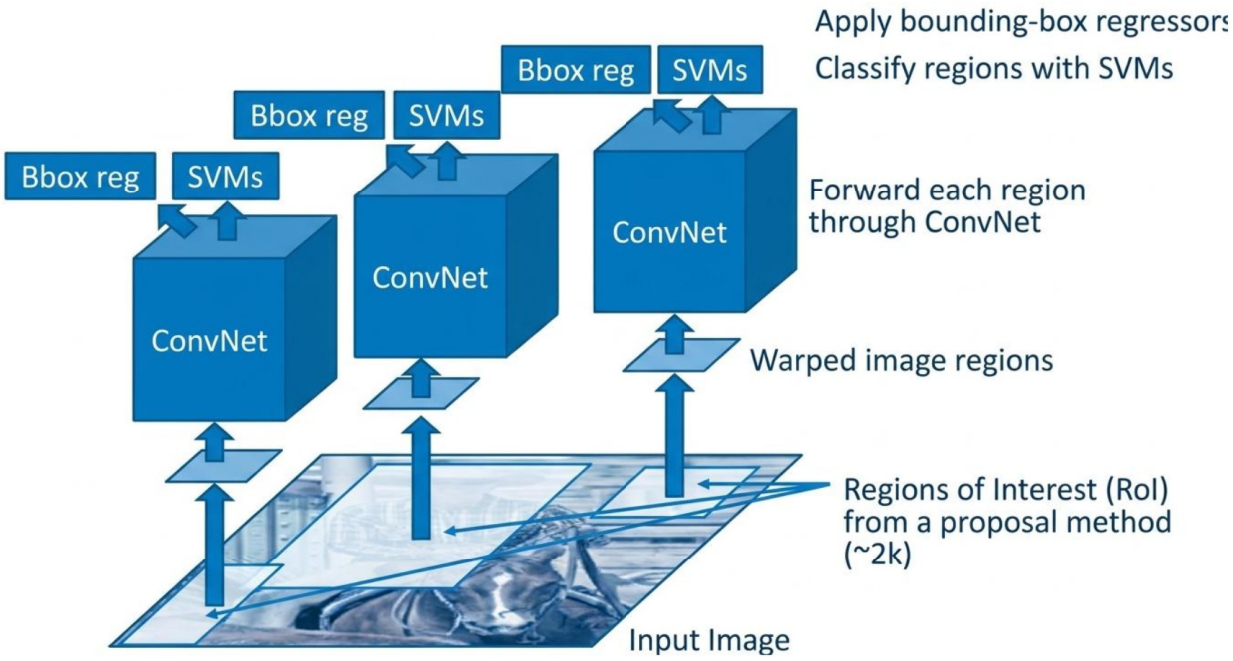
- Analyze Fast R-CNN and how single-pass feature extraction with RoI Pooling improves computational speed.
- Understand Faster R-CNN and how the Region Proposal Network (RPN) eliminates the selective search bottleneck.
- Implement end-to-end object detection using Faster R-CNN on the Blood Cell Count and Detection (BCCD) dataset.

2. Region-Based Convolutional Neural Networks (R-CNN Family)

2.1 R-CNN (Region-Based CNN)

R-CNN extracts a bunch of regions from the given image using selective search, and then checks if any of these boxes contains an object. We first extract these regions, and for each region, CNN is used to extract specific features. Finally, these features are then used to detect objects. Unfortunately, R-CNN becomes rather slow due to these multiple steps involved in the process.

Talha Shahid



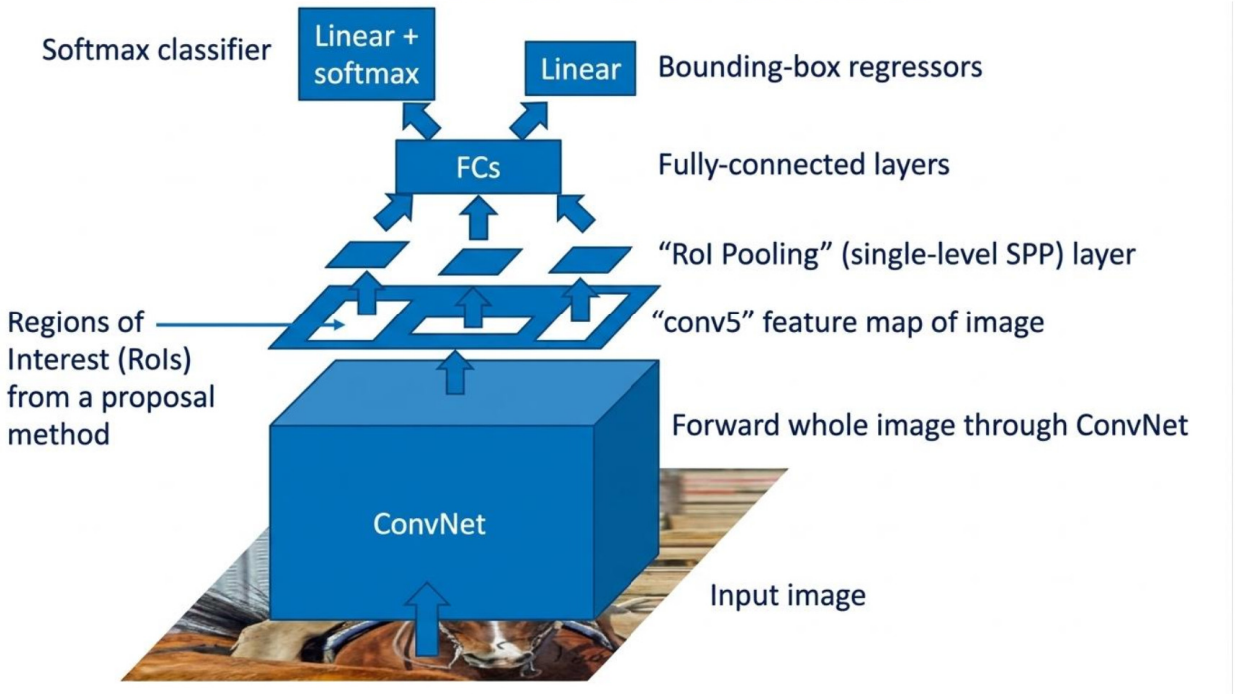
Input Image → Regions of Interest (RoI) from a proposal method (~2k regions via Selective Search) → Warped image regions → Forward each region independently through ConvNet → Classify regions with SVMs & Apply bounding-box regressors.

2.2 Fast R-CNN

Fast R-CNN, on the other hand, passes the entire image to ConvNet which generates regions of interest (instead of passing the extracted regions from the image). Also, instead of using three different models (as we saw in R-CNN), it uses a single model which extracts features from the regions, classifies them into different classes, and returns the bounding boxes.

All these steps are done simultaneously, thus making it execute faster as compared to R-CNN. Fast R-CNN is, however, not fast enough when applied on a large dataset as it also uses selective search for extracting the regions.

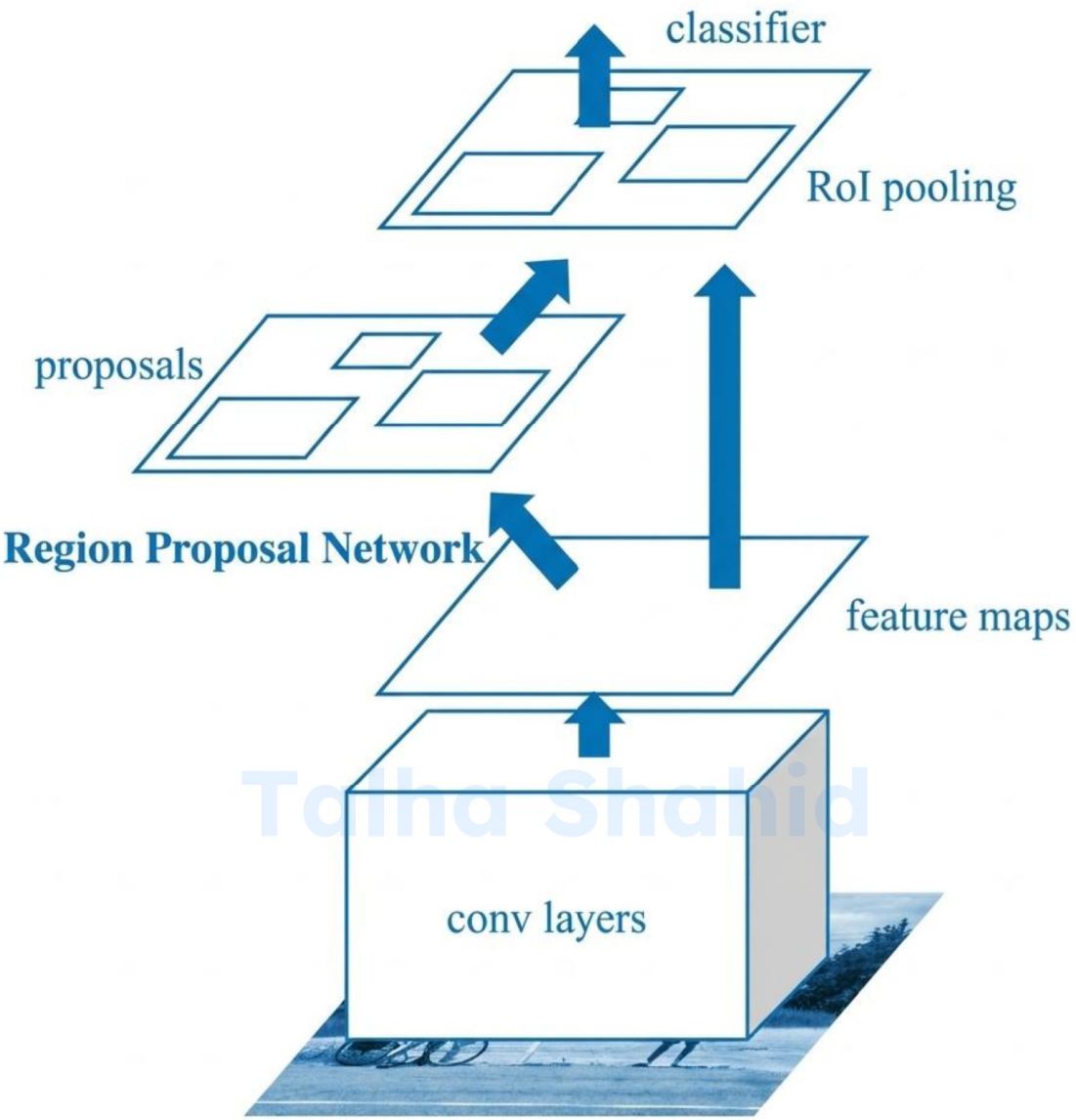
Talha Shahid



Input Image → Forward whole image through ConvNet → "conv5" feature map of image + Regions of Interest (RoIs) from a proposal method → "RoI Pooling" (single-level SPP) layer → Fully-connected layers (FCs) → Softmax classifier (Linear + softmax) & Bounding-box regressors (Linear).

Faster R-CNN fixes the problem of selective search by replacing it with Region Proposal Network (RPN). We first extract feature maps from the input image using ConvNet and then pass those maps through a RPN which returns object proposals. Finally, these maps are classified and the bounding boxes are predicted.

Talha Shahid



Input Image → Conv Layers → Feature Maps → Region Proposal Network (RPN) generates proposals → RoI Pooling combines feature maps and proposals → Classifier & Bounding-Box Regressor.

2.2.1 Faster R-CNN Algorithm Workflow

The steps followed by a Faster R-CNN algorithm to detect objects in an image:

- Take an input image and pass it to the ConvNet which returns feature maps for the image.
- Apply Region Proposal Network (RPN) on these feature maps and get object proposals.
- Apply ROI pooling layer to bring down all the proposals to the same size.
- Finally, pass these proposals to a fully connected layer in order to classify and predict the bounding boxes for the image.

2.3 Architectural Comparison Summary

Algorithm	Features	Prediction time / image	Limitations
CNN	Divides the image into multiple regions and then classifies each region into various classes.	–	Needs a lot of regions to predict accurately and hence high computation time.
R-CNN	Uses selective search to generate regions. Extracts around 2000 regions from each image.	40-50 seconds	High computation time as each region is passed to the CNN separately. Also, it uses three different models for making predictions.

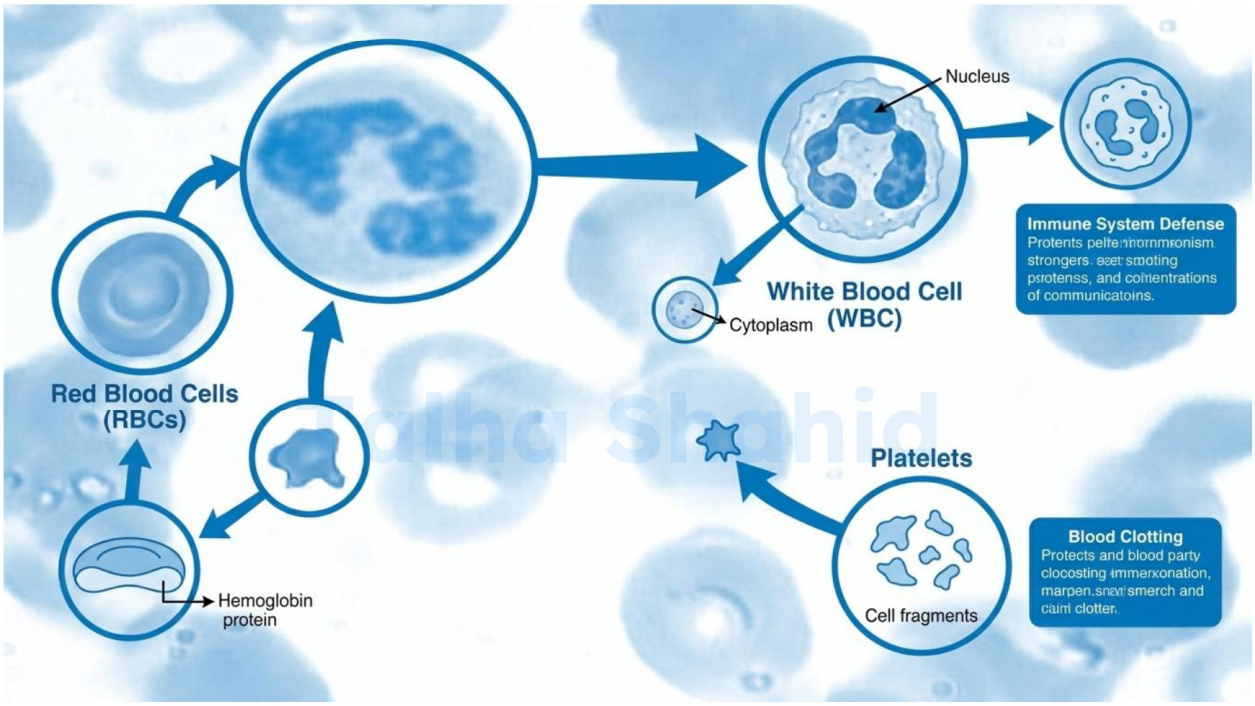
Talha Shahid

Algorithm	Features	Prediction time / image	Limitations
Fast R-CNN	Each image is passed only once to the CNN and feature maps are extracted. Selective search is used on these maps to generate predictions. Combines all the three models used in R-CNN together.	2 seconds	Selective search is slow and hence computation time is still high.
Faster R-CNN	Replaces the selective search method with region proposal network (RPN) which makes the algorithm much faster.	0.2 seconds	Object proposal takes time and as there are different systems working one after the other, the performance of systems depends on how the previous system has performed.

Talha Shahid

3. Understanding the Problem Statement

We will be working on a healthcare related dataset and the aim here is to solve a Blood Cell Detection problem. Our task is to detect all the Red Blood Cells (RBCs), White Blood Cells (WBCs), and Platelets in each image taken via microscopic image readings. Below is a sample of what our final predictions should look like:



The reason for choosing this dataset is that the density of RBCs, WBCs and Platelets in our blood stream provides a lot of information about the immune system and hemoglobin. This can help us potentially identify whether a person is healthy or not, and if any discrepancy is found in their blood, actions can be taken quickly to diagnose that.

Dataset Resource: [BCCD Dataset (Blood Cell Count and Detection)]

4. Data Exploration

4.1 Dataset Structure and Files

It's always a good idea to first explore the data we have. This helps us not only unearth hidden patterns, but gain a valuable overall insight into what we are working with. The three files created out of the entire dataset are:

- **train_images:** Images that we will be using to train the model. We have the classes and the actual bounding boxes for each class in this folder.
- **test_images:** Images in this folder will be used to make predictions using the trained model. This set is missing the classes and the bounding boxes for these classes.
- **train.csv:** Contains the name, class and bounding box coordinates for each image. There can be multiple rows for one image as a single image can have more than one object.

Talha Shahid

4.2 Loading and Inspecting the Annotation CSV

Let's read the `.csv` file (you can create your own `.csv` file from the original dataset if you feel like experimenting) and print out the first few rows. We'll need to first import the below libraries for this:

```
# importing required librariesimport pandas as pdimport matplotlib.pyplot as plt%matplotlib inlinefrom matplotlib import patches# read the csv file using read_csv function of pandastrain = pd.read_csv('train.csv')train.head()
```

Output (`train.head()`):

Index	image_names	cell_type	xmin	xmax	ymin	ymax
0	1.jpg	RBC	68	165	154	249
1	1.jpg	RBC	1	66	145	260
2	1.jpg	RBC	207	334	160	270
3	1.jpg	RBC	435	540	347	437
4	1.jpg	RBC	535	639	356	464

There are 6 columns in the `train` file. Let's understand what each column represents:

- **image_names:** contains the name of the image
- **cell_type:** denotes the type of the cell
- **xmin:** x-coordinate of the bottom left part of the image
- **xmax:** x-coordinate of the top right part of the image
- **ymin:** y-coordinate of the bottom left part of the image
- **ymax:** y-coordinate of the top right part of the image

Talha Shahid

4.3 Inspecting Training Images and Class Distribution

```
# reading single image using imread function of matplotlib
image = plt.imread('images/1.jpg')
plt.imshow(image)

# Number of unique training images
train['image_names'].nunique()
# So, we have 254 training images.

# Number of classes
train['cell_type'].value_counts()
```

Output (`value_counts()`):

Class	Count
RBC	2909
WBC	262
Platelets	252

(Name: cell_type, dtype: int64)

4.4 Visualizing Ground-Truth Bounding Boxes

We have three different classes of cells, i.e., RBC, WBC and Platelets. Finally, let's look at how an image with detected objects will look like:

Talha Shahid

```
fig = plt.figure()

# add axes to the image
ax = fig.add_axes([0, 0, 1, 1])

# read and plot the image
image = plt.imread('images/1.jpg')
plt.imshow(image)

# iterating over the image for different objects
for _, row in train[train.image_names == "1.jpg"].iterrows():
    xmin = row.xmin
    xmax = row.xmax
    ymin = row.ymin
    ymax = row.ymax

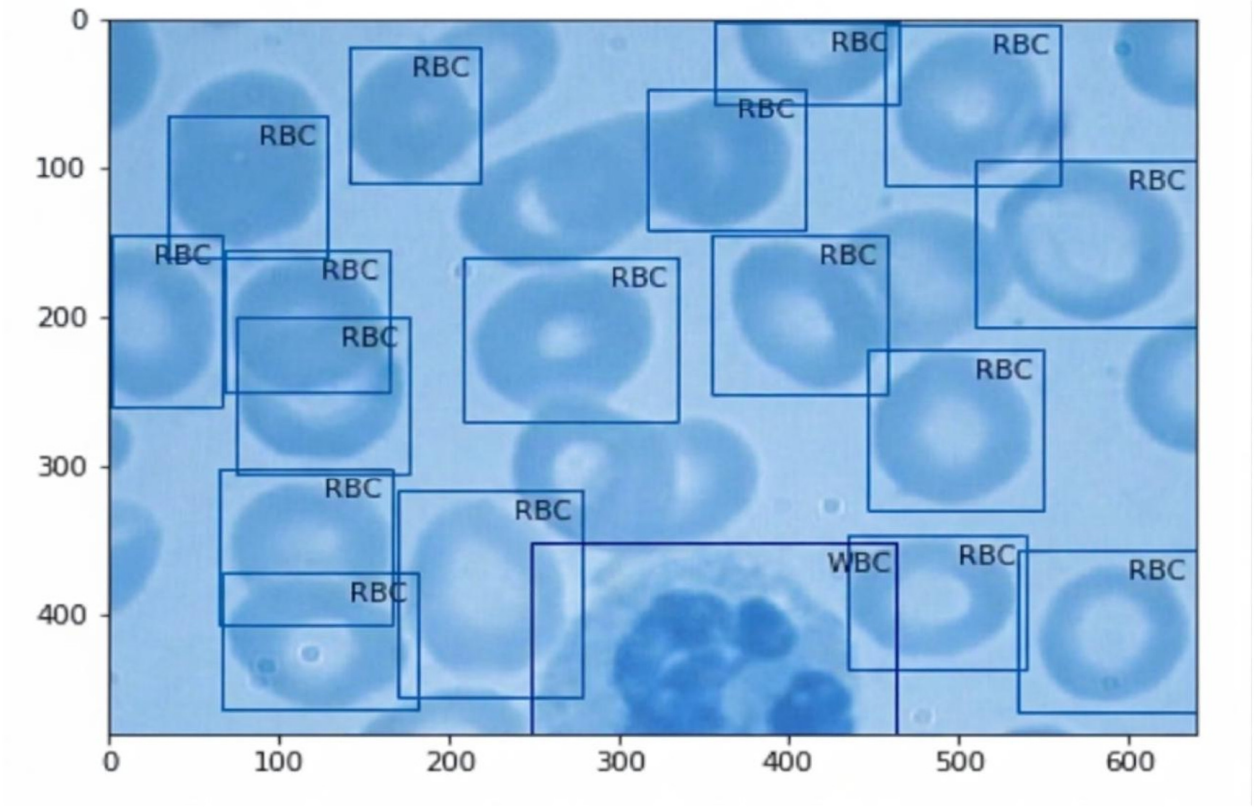
    width = xmax - xmin
    height = ymax - ymin

    # assign different color to different classes of objects
    if row.cell_type == 'RBC':
        edgecolor = 'r'
        ax.annotate('RBC', xy=(xmax - 40, ymin + 20))
    elif row.cell_type == 'WBC':
        edgecolor = 'b'
        ax.annotate('WBC', xy=(xmax - 40, ymin + 20))
    elif row.cell_type == 'Platelets':
        edgecolor = 'g'
        ax.annotate('Platelets', xy=(xmax - 40, ymin + 20))

    # add bounding boxes to the image
    rect = patches.Rectangle((xmin, ymin), width, height, edgecolor=edgecolor,
facecolor='none')

    ax.add_patch(rect)
```

Talha Shahid



This is what a training example looks like. We have the different classes and their corresponding bounding boxes. Let's now train our model on these images. We will be using the keras_frcnn library to train our model as well as to get predictions on the test images.

5. Implementing Faster R-CNN

5.1 Repository Setup and Input Annotation Format

1. Clone the keras-frcnn repository:

```
git clone https://github.com/kbardool/keras-frcnn.git
```

2. Move the train_images and test_images folder, as well as the train.csv file, to the cloned repository. In order to train the model on a new dataset, the format of the input should be:

```
filepath,x1,y1,x2,y2,class_name
```

where:

- **filepath:** is the path of the training image
- **x1:** is the xmin coordinate for bounding box
- **y1:** is the ymin coordinate for bounding box
- **x2:** is the xmax coordinate for bounding box
- **y2:** is the ymax coordinate for bounding box
- **class_name:** is the name of the class in that bounding box

5.2 Converting CSV Annotations to Text Format

3. We need to convert the `.csv` format into a `.txt` file which will have the same format as described above. Make a new dataframe, fill all the values as per the format into that dataframe, and then save it as a `.txt` file:

Talha Shahid

```
import pandas as pd

# Create a new DataFrame for the annotation format
data = pd.DataFrame()

# Copy the image names from the training data
data['format'] = train['image_names']

# Vectorized path prefixing (faster and cleaner than looping)
data['format'] = 'train_images/' + data['format']

# Vectorized string concatenation for coordinates and class names
# Note: keras-frcnn expects path,x1,y1,x2,y2,class_name order
data['format'] = (
    data['format'] + ',' +
    train['xmin'].astype(str) + ',' +
    train['ymin'].astype(str) + ',' +
    train['xmax'].astype(str) + ',' +
    train['ymax'].astype(str) + ',' +
    train['cell_type']
)

# Save to a text file without headers or row indices
data.to_csv('annotate.txt', header=None, index=None, sep=' ')
```

Talha Shahid

5.3 Training the Faster R-CNN Model

4. Train our model! We will be using the train_frcnn.py file to train the model:

```
cd keras-frcnnpython train_frcnn.py -o simple -p annotate.txt
```

5.4 Running Inference and Saving Predictions

5. It's prediction time! keras_frcnn makes the predictions for the new images and saves them in a new folder. We just have to make two changes in the test_frcnn.py file to save the images:

- Remove the comment from the last line of this file:

```
import pandas as pd

# Create a new DataFrame for the annotation format
data = pd.DataFrame()

# Copy the image names from the training data
data['format'] = train['image_names']

# Vectorized path prefixing (faster and cleaner than looping)
data['format'] = 'train_images/' + data['format']

# Vectorized string concatenation for coordinates and class names
# Note: keras-frcnn expects path,x1,y1,x2,y2,class_name order
data['format'] = (
    data['format'] + ',' +
    train['xmin'].astype(str) + ',' +
    train['ymin'].astype(str) + ',' +
    train['xmax'].astype(str) + ',' +
    train['ymax'].astype(str) + ',' +
    train['cell_type']
)

# Save to a text file without headers or row indices
data.to_csv('annotate.txt', header=None, index=None, sep=' ')
```

Talha Shahid

- Add comments on the second last and third last line of this file:

```
# cv2.imshow('img', img)
# cv2.waitKey(0)
```

6. Let’s make the predictions for the new images:

```
python test_frcnn.py -p test_images
```

6. Lab Task

6.1 Object Detection Using Regional Convolutional Neural Networks (R-CNN)

Note: Do not use the mentioned BCCD dataset; use another dataset of your choice.

Objective: Implement object detection using a Regional Convolutional Neural Network (R-CNN) on a dataset of your choice. R-CNN is a classic object detection method that combines region proposal, feature extraction, and object classification to identify and localize objects within images.

Talha Shahid